

AniFinal (AniFlow) -- Full Engineering Audit Report

Scope: Entire repository at anshsingh70007-hash/AniFinal, branch main (v1.8.2, versionCode 43). ~12,450 lines of Kotlin across 58 files, single :app module, Compose-only UI, Navigation 3, Media3/ExoPlayer, Ktor, Coil 3, DataStore, two product flavors (standard, redesign), phone + TV form factors.

Architecture mental model: MainActivity -> MainNavigation() (Nav3 backstack: Main/Detail/Player) -> MainScreen switches between 4 tabs x 2 flavors x 2 device types (16 screen permutations). Data comes from two remote sources: AniList GraphQL (catalog metadata) and api.anilight.live (episodes/streams), joined by fuzzy title search. Persistence is 3 DataStore-backed JSON-blob stores. No DI, no Room, no domain layer, no working tests.

PART 1 -- CRITICAL ISSUES

C-1. Unit and instrumentation tests reference classes that do not exist -- the test source sets cannot compile

- Severity: Critical
- Why: `app/src/test/java/com/example/aniflow/ui/main/MainScreenViewModelTest.kt` imports `com.example.aniflow.data.DataRepository` and references `viewModel.uiState / MainScreenUiState.Loading` -- none of these exist anywhere in the codebase. `MainScreenViewModelTest` also constructs `MainScreenViewModel(FakeMyModelRepository())` with 1 argument; the real constructor (`MainScreenViewModel.kt:14-19`) takes 4. `MainScreenTest.kt` (androidTest) calls `MainScreen(FAKE_DATA)` -- no such overload exists. Any `./gradlew test` or `connectedCheck` fails at compilation, meaning CI can never be enabled and no regression safety exists.
- Files: `app/src/test/java/com/example/aniflow/ui/main/MainScreenViewModelTest.kt` (whole file), `app/src/androidTest/java/com/example/aniflow/ui/main/MainScreenTest.kt` (whole file)
- Root cause: These are leftover template tests from the original project scaffold, never updated as the app evolved.
- Fix: Delete both files, then write real tests: a `MainScreenViewModelTest` using a fake `AnimeRepository` (the interface in `data/repository/AnimeRepository.kt` already exists and is perfectly mockable), asserting `trending/isLoading` state transitions with `kotlinx-coroutines-test StandardTestDispatcher`; a `PlayerViewModelTest` covering `handlePlaybackError` fallback logic (this is the most bug-prone code in the app and 100% pure logic -- ideal test target).
- Risks if unfixed: Zero regression protection; every future fix in this report can silently break something else.
- Difficulty: Easy (delete) / Medium (write real tests)
- Expected improvement: Working CI, safe refactoring for everything else in this report.

C-2. Release builds are signed with the debug keystore

- Severity: Critical
- Why: `app/build.gradle.kts:33` -- `signingConfig = signingConfigs.getByName("debug")`. Debug keystores are machine-local, auto-generated, and publicly known-password (`android/androidkeystore`). Consequences: (1) anyone can build an APK that installs over your users' installs as an "update" -- combined with your self-updater this is a supply-chain takeover vector; (2) building on another machine changes the signature and breaks updates for all existing users; (3) the app can never go to Play Store with this signature.
- Files: `app/build.gradle.kts:29-36`
- Root cause: Convenience shortcut to make `assembleRelease` work without keystore setup.
- Fix: Create a real release keystore, load credentials from `~/gradle/gradle.properties` or environment variables (never commit them):

```
signingConfigs {
```

Additionally, because you self-distribute via AppUpdater, verify the downloaded APK's signature matches the running app's signature before invoking the installer (see C-3).

- Risks if unfixed: Malicious "update" APK installation on user devices; permanent update breakage if the debug keystore is lost.
- Difficulty: Easy
- Expected improvement: Real update chain integrity; Play Store eligibility.

C-3. Self-updater downloads and installs APKs over an unauthenticated, unverified channel

- Severity: Critical
- Why: The update flow is: `DefaultAnimeRepository.checkUpdates()` (line 434-452) fetches `app_update.json` from `raw.githubusercontent.com` -> `AppUpdater.downloadAndInstall()` downloads the APK with plain `URLConnection` and immediately launches the installer. There is no checksum, no signature verification, no HTTPS pinning, and `AndroidManifest.xml:22` sets `android:usesCleartextTraffic="true"` with `network_security_config.xml` also permitting cleartext globally -- so an on-path attacker (open Wi-Fi, malicious DNS) can redirect the JSON `updateUrl` or the APK bytes themselves. Because the app also holds `REQUEST_INSTALL_PACKAGES`, this is remote code execution with one MITM.
- Files: `app/src/main/java/com/example/aniflow/utlis/AppUpdater.kt:20-83`, `app/src/main/java/com/example/aniflow/data/repository/DefaultAnimeRepository.kt:434-452`, `app/src/main/AndroidManifest.xml:6,22-23`, `app/src/main/res/xml/network_security_config.xml`
- Root cause: Update pipeline built for convenience; security requirements never designed in.
- Fix (in order of importance):
 1. Add "sha256" to `app_update.json` (and to `publish_update.py` generation); in `AppUpdater`, hash the downloaded file with `MessageDigest.getInstance("SHA-256")` and abort install on mismatch.
 2. Verify APK signing cert equals the running app's cert via `PackageManager.getPackageArchiveInfo(file, GET_SIGNING_CERTIFICATES)` before `installApk()`.
 3. Set `cleartextTrafficPermitted="false"` in `network_security_config.xml` and remove `android:usesCleartextTraffic="true"` -- every endpoint used (`graphql.anilist.co`, `api.anilist.live`, GitHub raw) is HTTPS. If some resolved stream URLs are HTTP, whitelist only those specific domains with a `<domain-config>` instead of a global opt-in.
- Risks if unfixed: Full device compromise of every user via network attacker; Play Protect flags.
- Difficulty: Medium
- Expected improvement: Closes the app's single largest attack surface.

C-4. WatchHistoryStore uses an unmanaged CoroutineScope with fire-and-forget writes -- race conditions and lost writes

- Severity: Critical
- Why: `WatchHistoryStore.kt:21` creates private `val scope = CoroutineScope(Dispatchers.IO)` (no `SupervisorJob`, never cancelled). `saveProgress()` (line 48-76) launches read-modify-write cycles into it: `getHistoryList()` -> `mutate` -> `saveHistory()`. Two problems: (1) Race condition: `PlayerScreen` calls `saveProgress` from the pause listener (`PlayerScreen.kt:212-221`), the 15s heartbeat (line 346-366), and `onDispose` (line 235-244). Two concurrent launches both read the same old list, both write -- last-writer-wins destroys the other's entry. (2) The scope has no `SupervisorJob`, so a single uncaught exception in one write kills the scope and all future history writes silently stop for the app session. (3) Writes in flight when the process dies are lost with no flush.
- Files: `app/src/main/java/com/example/aniflow/data/WatchHistoryStore.kt:21,48-97`
- Root cause: `DataStore`'s transactional `edit {}` is being bypassed by doing read-then-write across two separate suspend calls, wrapped in unstructured concurrency.
- Fix: Make `saveProgress/removeHistory/clearHistory` suspend and do the read-modify-write inside a single `dataStore.edit {}` block (`DataStore` serializes edits, giving atomicity for free):

```
suspend fun saveProgress(...) {
```

cont

Call it from `viewModelScope` in `PlayerViewModel.saveProgress` (which currently just delegates non-suspending). For the

- onDispose save (can't suspend there), route through the ViewModel: `viewModelScope.launch { store.saveProgress(...) } -- viewModelScope.outlives.composable.disposal`.
- Risks if unfixed: "Continue Watching" randomly loses/corrupts progress; history writes die silently mid-session.
- Difficulty: Medium
- Expected improvement: Reliable watch history -- a core feature of the app.

C-5. Hardcoded fallback video (w3schools.com/html/mov_bbb.mp4) plays when stream resolution fails

- Severity: Critical (user-facing correctness)
- Why: `DefaultAnimeRepository.getStreamingSources()` (lines 365-381) returns Big Buck Bunny from w3schools plus a random GitHub-hosted VTT when the real provider fails. The user taps an episode of Attack on Titan and gets a cartoon rabbit labeled "1080p (Fallback)" with no explanation. This masks every provider failure as "playback of the wrong video," making error reports undiagnosable, and hammers a third-party site you don't control.
- Files: `app/src/main/java/com/example/aniflow/data/repository/DefaultAnimeRepository.kt:361-381`
- Root cause: Debug scaffolding left in production.
- Fix: Return `EpisodeSourcesResponse(sources = emptyList())` and let `PlayerViewModel.loadStreamingSourcesForIndex` set a real error state ("No streams found for this episode -- the provider may not have it yet") with `Retry + episode-list navigation`. `PlayerViewModel` already has `hasError/errorMessage` plumbing; you just need `if (sources.sources.isEmpty()) { hasError.value = true; errorMessage.value = ... }`.
- Risks if unfixed: Users think the app is broken/hacked; support burden; dependency on w3schools uptime.
- Difficulty: Easy
- Expected improvement: Honest, actionable error UX; diagnosable provider failures.

C-6. Theme system mutates global mutableStateOf vars -- broken Material integration, race-prone, non-idiomatic

- Severity: Critical (architectural)
- Why: `theme/Color.kt` declares top-level var `PrimaryDark` by `mutableStateOf(...)` etc., and `Theme.kt:28-49` assigns them during composition (if `(isLight) { PrimaryDark = ...; TextPrimary = ... }`). This is a side effect in the composition phase -- Compose explicitly forbids writing state during composition; it can cause an extra recomposition pass each frame and inconsistent frames where half the globals are light and half dark. It also means every one of the ~50 usages of `TextPrimary/SurfaceCard` across all screens bypasses `MaterialTheme.colorScheme`, so Material components (`Slider`, `NavigationBar`, `Dialogs`) and your custom UI can disagree. Light theme is also incoherent: `Theme.kt:52` always builds `darkColorScheme(...)` even in light mode, so `colorScheme.surface/onSurface` are dark-scheme semantics with light values.
- Files: `app/src/main/java/com/example/aniflow/theme/Color.kt` (entire file), `app/src/main/java/com/example/aniflow/theme/Theme.kt:26-49`
- Root cause: Global mutable color singletons chosen instead of Compose's theming system.
- Fix: Delete the var globals. Define three immutable `ColorSchemes` (`dark`, `amoled`, `light` with `lightColorScheme()`), select by `themeMode` in `AniFlowTheme`, and reference `MaterialTheme.colorScheme.*` in screens. For non-Material tokens (e.g., `SuccessGreen`, glass tokens) create an immutable `AniFlowColors` data class provided via a `staticCompositionLocalOf`. This is a large but mechanical migration (find/replace `TextPrimary -> MaterialTheme.colorScheme.onBackground`, etc.).
- Risks if unfixed: Flash of wrong-theme frames, subtle recomposition storms, impossible to ever support dynamic color / proper light mode.
- Difficulty: Hard (touches all screens)
- Expected improvement: Correct theming, fewer recompositions, unlocks Material 3 dynamic color.

C-7. Repository, stores, and API clients constructed inside composition -- no DI, state lost on process death, untestable

- Severity: Critical (architectural)
- Why: `Navigation.kt:19-22` does `remember { DefaultAnimeRepository(context.applicationContext) }` etc. inside a composable. Consequences: (1) all in-memory caches in `DefaultAnimeRepository` (lines 22-47: eight cached lists) are tied to the composition -- activity recreation (e.g., theme change, process death) rebuilds everything; (2) `MainScreen.kt:76-81` has a default parameter that constructs a `ViewModel` with duplicate store instances (`WatchlistStore(context)` at line 78 and again at line 104 `remember { watchlistStore ?: WatchlistStore(context) }`) -- two `WatchlistStore` objects wrapping the same `DataStore` is

harmless only by luck; (3) MainScreenViewModel takes a raw android.content.Context (line 18) -- a ViewModel holding a Context is a leak-by-design pattern (here it's applicationContext, so it survives, but nothing enforces that); (4) nothing is injectable -> nothing is testable.

- Files: app/src/main/java/com/example/aniflow/Navigation.kt:17-27, app/src/main/java/com/example/aniflow/ui/main/MainScreen.kt:73-109, app/src/main/java/com/example/aniflow/ui/main/MainScreenViewModel.kt:14-21
- Root cause: No dependency injection framework or composition root.
- Fix: Add a minimal composition root -- either Hilt (@HiltAndroidApp Application, @Singleton bindings for AnimeRepository, stores, HttpClient) or, if you want zero new dependencies, a hand-rolled AppContainer on a custom Application class (class AniFlowApp : Application { val container = AppContainer(this) }) and read it via LocalContext.current.applicationContext as AniFlowApp. Use factories: viewModel { MainScreenViewModel(container.repository, ...) }. Replace the Context in MainScreenViewModel with the two things it actually needs: SettingsStore and a currentVersionCode: Int provider.
- Risks if unfixed: Cache thrash on recreation, untestable ViewModels, duplicate store instances drifting.
- Difficulty: Medium-Hard
- Expected improvement: Testability, correct singleton lifecycles, groundwork for every other refactor.

C-8. Proguard keeps the entire data package and a nonexistent "Stormbreaker" class; R8 obfuscation effectively disabled where it matters

- Severity: High (listed under Critical because it also hides a real risk: -overloadaggressively + broad keeps can break serialization in subtle ways)
- Why: proguard-rules.pro:
 - -keep class com.example.aniflow.security.Stormbreaker { *; } -- no such class or package exists in the source tree. Dead rule indicating copy-paste from another project.
 - -keep class com.example.aniflow.data. { *; } and -keep class io.ktor. { *; } keep everything -- the whole data layer and the entire Ktor library escape shrinking/obfuscation. Ktor ships consumer rules already; keeping all of io.ktor.** inflates the APK significantly.
 - -overloadaggressively is known to break libraries using reflection and gives negligible size win -- risky flag with no benefit here.
- Files: app/proguard-rules.pro (entire file)
- Root cause: Shotgun keep rules added to fix a past R8 crash instead of targeted rules.
- Fix: Delete the Stormbreaker rule and -overloadaggressively. Replace the blanket data keep with only what kotlinx-serialization needs (the @Serializable/\$\$serializer rules you already have are sufficient -- modern kotlinx-serialization + R8 works with just the bundled rules). Remove -keep class io.ktor. { *; }; keep only -dontwarn io.ktor. if needed. Then test a release build end-to-end (search, episode load, playback, history).
- Risks if unfixed: Bloated APK (~megabytes of unshrunk Ktor), trivial reverse engineering of the provider/decrypt logic in AniLightProvider (your "raye" XOR scheme at AniLightProvider.kt:400-417 is plaintext-readable in any decompiler anyway, but currently it isn't even renamed).
- Difficulty: Easy-Medium (needs release regression pass)
- Expected improvement: Smaller APK, actual obfuscation, removal of a footgun flag.

C-9. Binary APKs (~250 MB+) and logcat dumps committed to the git repository

- Severity: Critical (repo hygiene / secrets)
- Why: The repo is 593 MB. releases/ and releases/backup/ contain ~14 APKs (56 MB each for redesign builds); logcat.txt (676 KB), logcat_pid.txt, logcat_pid_utf8.txt are committed at the root. Logcat dumps can contain device identifiers, tokens, and stream URLs (your own code logs full JSON bodies -- see H-7). Git history keeps every APK forever; clones are painfully slow; GitHub raw is being used as your APK CDN (app_update.json points into the repo).
- Files: /releases/*, /logcat.txt, .gitignore
- Root cause: Using the source repo as a release CDN + debugging artifacts committed.
- Fix: Move APK distribution to GitHub Releases (assets, not tracked files) and update publish_update.py + app_update.json updateUrl accordingly. Delete releases/ and logcat files, add releases/, .apk, logcat.txt to .gitignore, and rewrite history with git filter-repo if you care about clone size.
- Risks if unfixed: Repo becomes unusable at scale; potential PII leakage in logs; raw.githubusercontent bandwidth throttling breaking updates.
- Difficulty: Easy (going forward) / Medium (history rewrite)

- Expected improvement: ~550 MB repo shrink, proper release channel.

PART 2 -- HIGH SEVERITY ISSUES

H-1. PlayerScreen stale-closure bug: player listener captures currentEp from an old composition

- Severity: High
- Why: PlayerScreen.kt:194-244 -- val currentEp = episodeList.getOrNull(currentEpisodeIndex) is computed in composition, then captured by the Player.Listener created inside DisposableEffect(exoPlayer). The effect keys only on exoPlayer (which is remember(animeld)), so the listener object is created once and never recreated -- DisposableEffect re-runs only when keys change. The listener therefore holds currentEp from the first composition (usually null, since the episode list loads async). Result: onIsPlayingChanged's save-on-pause (lines 214-220) and the onDispose save (lines 237-241) use a stale/null currentEp -- early pauses save nothing, and after switching episodes the dispose-save can write progress against the wrong episode.
- Files: app/src/main/java/com/example/aniflow/ui/player/PlayerScreen.kt:194-244
- Root cause: Reading composition state inside a non-recomposing effect without rememberUpdatedState.
- Fix: Inside the effect, read the ViewModel's live state instead of captured composition values:

```
val ep = viewModel.episodeList.value.getOrNull(viewModel.currentEpisodeIndex.value)
```

(the ViewModel StateFlows are always current), or wrap with val currentEpState = rememberUpdatedState(currentEp) and read currentEpState.value inside the listener/onDispose.

- Risks: Lost/incorrect resume positions -- the top complaint category for streaming apps.
- Difficulty: Easy
- Expected improvement: Correct progress saving in all lifecycle paths.

H-2. No lifecycle pause, no onStop release -- player keeps playing audio in background without a MediaSession

- Severity: High
- Why: ExoPlayer.Builder(...).setAudioAttributes(audioAttributes, true) (PlayerScreen.kt:188) enables focus acquisition, which is good. But: (1) there is no lifecycle observer -- when the user presses Home or the screen locks, the player continues playing (audio keeps going with no notification, no MediaSession, no way to pause except reopening the app). The media3-session dependency is declared in build.gradle.kts but never used anywhere (verified by import scan). (2) android.supportsPictureInPicture="true" is declared in the manifest but there is no enterPictureInPictureMode call or PiP params anywhere -- the declaration is dead. This is also a Play policy problem: background media playback without a MediaSession/notification.
- Files: app/src/main/java/com/example/aniflow/ui/player/PlayerScreen.kt:145-192,196-244, app/src/main/AndroidManifest.xml:31, app/build.gradle.kts (media3.session)
- Root cause: Player lifecycle wired only to composition, not to app lifecycle.
- Fix: Minimum: add a LifecycleEventObserver (via LocalLifecycleOwner) -- ON_STOP -> exoPlayer.pause() (and save progress), ON_START -> optionally resume. Better: implement PiP properly (call activity.enterPictureInPictureMode(PictureInPictureParams) on onUserLeaveHint/ON_PAUSE while playing, with 16:9 aspect ratio) since the manifest already claims support. If you want true background audio, add MediaSession + MediaSessionService using the already-declared media3-session artifact; otherwise remove that dependency.
- Risks: Battery drain, phantom audio, Play policy violation, confused users.
- Difficulty: Medium
- Expected improvement: Correct pause/resume behavior; real PiP; honest manifest.

H-3. Position polling loop and progress slider design cause seek-fighting and drift

- Severity: High
- Why: Two related defects in PlayerScreen.kt:
 1. The polling loop (lines 346-366) only updates currentPosition if (exoPlayer.isPlaying) -- when paused, the UI position freezes even after seeks from other code paths; and with controls hidden it polls every 2000ms, so on showing controls the

slider jumps.

2. `ScopedProgressSlider` (lines 898-923) calls `exoPlayer.seekTo(it.toLong())` on every `onValueChange` tick while dragging. That issues dozens of seeks per second on an HLS stream -- each seek can trigger buffering, so scrubbing stutters violently. Media players must seek on `onValueChangeFinished`, previewing locally during the drag.

- Files: `app/src/main/java/com/example/aniflow/ui/player/PlayerScreen.kt:346-366,898-923`

- Root cause: Slider bound directly to the player instead of a local drag state.

- Fix:

```
var dragPosition by remember { mutableStateOf<Float?>(null) }
```

Slider (

Also drop the `isPlaying` guard in the poll loop (update position whenever `playbackState == READY`), or better, replace polling entirely with `Player.Listener.onEvents` + a single 500ms ticker active only while controls are visible.

- Risks: Janky scrubbing (every user touches this), excessive network churn on HLS.

- Difficulty: Easy

- Expected improvement: Smooth scrubbing -- one of the most perceptible UX wins available.

H-4. `handlePlaybackError` resume position is lost when switching to a backup source

- Severity: High

- Why: When a source fails mid-playback and `PlayerViewModel.handlePlaybackError` swaps `selectedSource`, the `LaunchedEffect(selectedSource)` (`PlayerScreen.kt:250-323`) reads `val currentPos = exoPlayer.currentPosition` -- but by the time the error fired, `ExoPlayer` may already be in `STATE_IDLE` where `currentPosition` can be reset, and after `exoPlayer.stop()` at line 257 the resume logic depends on the pre-stop read being valid. Worse: `onPlayerError` for post-start failures (line 225-228) only shows an error and does not attempt backup URLs at all, even though `backupUrls` exist precisely for this -- the `hasPlayStarted` flag intentionally disables mid-play failover, so a CDN hiccup at minute 18 forces manual intervention.

- Files: `app/src/main/java/com/example/aniflow/ui/player/PlayerScreen.kt:222-229,250-323;`
`app/src/main/java/com/example/aniflow/ui/player/PlayerViewModel.kt:141-183`

- Root cause: Failover state machine split across the View (screen) and ViewModel with implicit ordering assumptions.

- Fix: Record position in the ViewModel at error time: in `onPlayerError`, capture `viewModel.lastKnownPosition = viewModel.currentPosition.value` (the poll loop keeps it fresh) and use that in the source-switch effect instead of a post-hoc `exoPlayer.currentPosition` read. Allow mid-play failover for network-category errors (`PlaybackException.errorCode` in `ERROR_CODE_IO_*`) while keeping the manual path for decoder errors.

- Risks: Users lose their place on every server failover; unrecoverable mid-play errors.

- Difficulty: Medium

- Expected improvement: Seamless failover with position retention -- big perceived-reliability win.

H-5. Repository caches are per-instance and screen flows never refresh -- Flow misuse (single-shot flow {} + first())

- Severity: High

- Why: Every repository read (`DefaultAnimeRepository.kt:69-246`) is a cold flow `{ emit(once) }` consumed by `first()` in the ViewModel -- i.e., these are suspend functions cosplaying as Flows. The eight cache fields + timestamps (lines 26-47) are duplicated boilerplate, not thread-safe (plain vars written from `Dispatchers.IO` flows without synchronization -- visibility not guaranteed), and reset whenever the composition-scoped repository (C-7) is recreated. There's no reactive refresh: pull-to-refresh doesn't exist, and the only refresh path is the 15-minute loop in `MainScreenViewModel.startPeriodicRefresh()` which only covers the schedule.

- Files: `app/src/main/java/com/example/aniflow/data/repository/DefaultAnimeRepository.kt:26-246,`
`app/src/main/java/com/example/aniflow/ui/main/MainScreenViewModel.kt:82-96,260-380`

- Root cause: API shape chosen before the data-flow design.

- Fix: Either make them honest suspend fun `getTrending(): List<Anime>` (simplest -- the callers already `.first()`), and extract a

generic memoizer:

```
private class CachedList<T>(private val ttl: Long, private val fetch: suspend () -> List<T>) {
```

priv

which collapses ~180 lines of duplication into one class. Longer-term, back home content with a small Room/DataStore cache for offline cold-start.

- Risks: Data races on cache fields, cache loss on recreation, dead-end for offline support.
- Difficulty: Medium
- Expected improvement: -150 LOC, thread-safe caching, honest API surface.

H-6. MainScreenViewModel.loadData() is a 120-line staggered-delay orchestration with fallback-detection by magic ID

- Severity: High
- Why: MainScreenViewModel.kt:260-380: hand-tuned delay(500L) staggering, a retry heuristic that checks `_trending.value.firstOrNull()?.id == 1535` (Death Note's AniList ID -- i.e., "does the data look like our hardcoded fallback"), and nested launch/join batches. Magic-ID detection breaks the moment fallback content changes; delays are guesses about AniList rate limits which AniListApi already handles with 429-aware retry (AniListApi.kt:57-95). Fallback data itself (`getFallbackAnimeList`, `DefaultAnimeRepository.kt:383-419`) leaks into real UI as if it were live content (users see a permanent fake "Death Note / One Piece / AoT" home page when offline, with `picsum.photos` random banner images -- line 389).
- Files: `app/src/main/java/com/example/aniflow/ui/main/MainScreenViewModel.kt:260-380`, `app/src/main/java/com/example/aniflow/data/repository/DefaultAnimeRepository.kt:383-432`
- Root cause: Fallback content conflated with error state; rate limiting handled in two layers.
- Fix: Model each section as sealed interface `SectionState { Loading; Success(list); Error }`. Remove fallback lists entirely; render explicit per-row error/retry UI (empty rows collapse -- the redesign home already hides empty sections). Replace magic-ID retry with a normal retry on Error state. Keep AniList throttling in one place: a shared `Semaphore(2)` inside `AniListApi` limits concurrency without brittle delay(s).
- Risks: Fake content presented as real; unmaintainable startup logic.
- Difficulty: Medium
- Expected improvement: Correct offline UX, ~80 fewer lines, single-source rate limiting.

H-7. Verbose production logging of response bodies and update JSON; 26 printStackTrace() calls

- Severity: High (privacy/perf)
- Why: `DefaultAnimeRepository.kt:438` logs the full update JSON on every launch; `AniListApi.kt:83` logs error bodies; `AniLightProvider` logs full exception chains per provider per episode; 26 `printStackTrace()` calls bypass Log entirely (unredactable, unfilterable). None of it is gated by `BuildConfig.DEBUG` -- and `buildConfig = false` (`app/build.gradle.kts:44`) means you can't even gate it without re-enabling `BuildConfig`.
- Files: `grep android.util.Log|printStackTrace` across `app/src/main` -- notably `DefaultAnimeRepository.kt`, `MainScreenViewModel.kt:158-170`, `AniListApi.kt:70-95`, all stores.
- Root cause: Debug logging accreted without a logging policy.
- Fix: Re-enable `buildConfig = true`, wrap a tiny `AppLog` object gating on `BuildConfig.DEBUG`, replace all `printStackTrace()` with `AppLog.e(TAG, msg, e)`. Never log payload bodies in release.
- Risks: Stream URLs/user viewing habits in logcat (readable by adb/other privileged apps), string allocation overhead on hot paths.
- Difficulty: Easy
- Expected improvement: Privacy hygiene, cleaner release logcat.

H-8. AniLightProvider.getStreamUrl: synchronized blocks in coroutines + redundant fetches + 13-way fan-out

- Severity: High
- Why: (1) `processSourcesResponse` (`AniLightProvider.kt:307-388`) does `synchronized(resolvedSources)` around

network-adjacent processing while its callers are coroutines with `withTimeoutOrNull` -- blocking a thread inside a synchronized block from up to ~13 parallel coroutines serializes them and blocks IO threads (thread starvation risk under load). The lists are already `Collections.synchronizedList`, making the extra lock redundant for element adds. (2) `getStreamUrl` re-fetches `$baseUrl/watch/$slug` (line 186-193) even though `getEpisodeList` fetched the same document moments earlier -- an entire redundant round trip on the critical play-tap -> first-frame path. (3) The dual sub/dub fan-out fires up to 13 concurrent requests per play tap against a free API -- high ban risk; and results race such that the "primary" source per quality group (`groupBy { it.quality } -> first()`, lines 276-282) is nondeterministic -- whichever provider responded first wins, so playback reliability varies run to run.

- Files: `app/src/main/java/com/example/aniflow/data/AniLightProvider.kt:173-305,307-388`
- Root cause: Java-style locking mixed into coroutine code; no session-level caching of the watch document.
- Fix: Have each provider coroutine build its own local `List<StreamingSource>`; collect with `awaitAll()` on `async` and merge once (no shared mutable state, no locks). Cache the `AniLightWatchResponse` per slug for ~5 min. Rank merged sources deterministically by a provider-priority list rather than arrival order. Cap concurrency with a `Semaphore(4)`.
- Risks: Slow stream resolution, IO-pool starvation, provider bans, nondeterministic playback quality.
- Difficulty: Medium
- Expected improvement: Faster time-to-first-frame, deterministic source selection, fewer requests.

H-9. Eight-plus unused dependencies shipped in the APK

- Severity: High (APK size / attack surface)
- Why: Import scanning across `app/src/main` shows zero usages of: `tv-foundation`, `tv-material` (`androidx.tv`), `lottie-compose`, `orbital`, `konfetti-compose`, `haze` + `haze-materials`, `androidx-palette`, `media3-session`, and `ktor-client-logging`. All are implementation deps in `app/build.gradle.kts:100-131`. R8 removes unused code, but keep-rule interactions (your blanket keeps, C-8) and resources still leak size; and every dependency is supply-chain surface. The "TV" screens are built with plain `Material3` + focus modifiers -- the TV Compose artifacts are dead weight.
- Files: `app/build.gradle.kts:100-131`, `gradle/libs.versions.toml`
- Root cause: Dependencies added for the "redesign" experiment (`haze/lottie/orbital/konfetti`) but the final implementation used hand-rolled glass modifiers instead.
- Fix: Remove all nine from `build.gradle.kts` and the catalog. If you later want real blur, re-add `haze` deliberately (see UI review -- your current "glass" is not blurred at all).
- Risks: Larger APK, slower builds, dependency-update churn for code you don't run.
- Difficulty: Easy
- Expected improvement: Faster builds, smaller APK, honest dependency list.

H-10. 51 MB of intro videos in the main source set -- shipped to BOTH flavors

- Severity: High
- Why: `app/src/main/res/raw/intro_first.mp4` (34 MB) + `intro_second.mp4` (16 MB) live in main, but `IntroOverlay` only runs when `packageName.endsWith(".redesign")` (`MainActivity.kt:38`). The standard flavor ships 51 MB of video it never plays. That's why redesign APKs are 56 MB. Also `isShrinkResources` cannot remove them because the resource IDs are referenced.
- Files: `app/src/main/res/raw/`, `app/src/main/java/com/example/aniflow/ui/redesign/components/IntroOverlay.kt:67-68`
- Root cause: Flavor-specific assets placed in the shared source set.
- Fix: Move both files to `app/src/redesign/res/raw/`. Guard the `R.raw` references so standard compiles (the `IntroOverlay` composable itself can move to the redesign source set, with flavor-specific `MainActivity` content or a no-op stub in standard). Consider recompressing the videos (H.265 or lower bitrate -- an intro doesn't need 34 MB) or replacing with a `Lottie/Compose` animation for ~100 KB.
- Risks: Bloated downloads, hosting costs, user install friction.
- Difficulty: Easy-Medium
- Expected improvement: -51 MB standard APK; potentially -40 MB redesign after recompression.

H-11. Flavor branching by `packageName.endsWith(".redesign")` string check scattered across 5+ files

- Severity: High (architecture)
- Why: `MainActivity.kt:38`, `Navigation.kt:24-26`, `MainScreen.kt:107-109`, `PlayerScreen.kt:119`, `MainScreen.kt:540` all re-derive `isRedesign` from the package name. This is fragile (an `appld` change silently flips UI), duplicated, and defeats the purpose of

product flavors -- flavor-specific source sets exist for exactly this. You even already have `app/src/redesign/res/values/config.xml` and `app/src/standard/res/values/config.xml` scaffolded but unused by code.

- Files: listed above; `app/src/{standard,redesign}/res/values/config.xml`
- Root cause: Runtime branching chosen over build-time source sets.
- Fix: Simplest: a bool resource in each flavor's `config.xml` (`<bool name="is_redesign">true</bool>`) read once and provided via `staticCompositionLocalOf<Boolean>` (`LocalsRedesign`) in `MainActivity`. Best: move flavor-specific composables into flavor source sets so dead UI isn't compiled into the other flavor at all (the standard APK currently contains the entire redesign UI and vice versa).
- Risks: Silent flavor mix-ups, doubled UI code in both APKs.
- Difficulty: Medium
- Expected improvement: Single source of truth; smaller APKs; safer appId changes.

H-12. DetailScreen episode-loading concurrency bug: nested launch inside collect, no failure handling

- Severity: High
- Why: `DetailScreen.kt:51-65`: inside `LaunchedEffect(animeId)`, it collects `getAnimeDetail` (a single-emission flow) and then launches a sibling coroutine on captured scope (`val scope = this`) to fetch episodes. If `getEpisodes` throws (it can't currently -- it swallows via `printStackTrace`, `DefaultAnimeRepository.kt:319-321`), `isLoading` stays true forever. If episodes come back empty (provider matched nothing), the screen shows the detail with an episode section that's just absent, with no "not available on provider" message. Same pattern duplicated in `RedesignDetailScreen.kt`.
- Files: `app/src/main/java/com/example/aniflow/ui/detail/DetailScreen.kt:51-65`, `app/src/main/java/com/example/aniflow/ui/redesign/RedesignDetailScreen.kt` (equivalent block)
- Root cause: Screen-level state machine written ad hoc in the composable instead of a `DetailViewModel`.
- Fix: Introduce `DetailViewModel` with data class `DetailUiState(val anime: Anime?, val episodes: List<Episode>, val episodesState: Loading|Empty|Error|Success, ...)`; sequential `val detail = repo.getAnimeDetail(id).first()`; `val eps = repo.getEpisodes(...)` inside one `viewModelScope.launch` with `try/catch/finally` setting state. Both Detail screens consume the same `ViewModel` -- removes duplication too.
- Risks: Infinite spinners, silent "no episodes" confusion (this is the "classic series matching" issue your own commit 90d73e7 was fighting).
- Difficulty: Medium
- Expected improvement: Robust detail loading; shared logic between both flavors' detail screens.

H-13. Fuzzy title matching to the streaming provider can silently play the wrong show

- Severity: High (correctness)
- Why: `DefaultAnimeRepository.getEpisodes()` (lines 264-324): if no `anilistId` match is found in provider search results, it falls back to `searchResults.firstOrNull()` (line 312) -- the first result for a cleaned title. `cleanTitleForSearch` strips everything after : and "Season N" -- so "Fate/Zero" vs "Fate/stay night", or any franchise with many entries, can bind episodes of the wrong series with zero indication to the user. Progress then gets saved against the AniList ID of the show the user thinks they're watching.
- Files: `app/src/main/java/com/example/aniflow/data/repository/DefaultAnimeRepository.kt:253-324`
- Root cause: No confidence threshold on the fallback match.
- Fix: Score candidates (normalized Levenshtein or token-set ratio between provider title and both AniList titles); accept only above a threshold (e.g., 0.75); below it, surface a "Select the matching series" sheet listing provider results with posters (you already get `posterUrl` in `ProviderSearchResult`), and persist the confirmed slug mapping (`DataStore` map `anilistId -> slug`) so it's a one-time choice. The persisted mapping also removes the repeated search on every detail open.
- Risks: Playing wrong content is the worst correctness failure a catalog app can have.
- Difficulty: Medium
- Expected improvement: Correct matches, user-controllable overrides, faster repeat loads.

H-14. TV form factor: seekbar disabled, no D-pad seek preview, focus managed via bare try/catch requestFocus

- Severity: High (TV UX)
- Why: `ScopedProgressSlider` sets `enabled = deviceType == DeviceType.PHONE` (`PlayerScreen.kt:920`) -- TV users cannot

scrub at all except +/-10s taps. Focus is managed by try { requestFocus() } catch in three places with delay(100) hacks (PlayerScreen.kt:375-390); on TV, showing controls steals focus to Play/Pause, and hiding sends it to an invisible Box -- pressing D-pad center with controls hidden shows controls (good) but the KEYCODE_BACK handling in onKeyEvent (line 447-450) intercepts back to hide controls, meaning back-press behavior depends on a race between focus moves. Additionally the volume state (currentVolume/maxVolume, lines 115-117) is read into state but never rendered or updated anywhere -- dead code.

- Files: app/src/main/java/com/example/aniflow/ui/player/PlayerScreen.kt:104-117,368-454,898-923
- Root cause: TV interaction model retrofitted onto a touch-first player.
- Fix: Implement D-pad seek on the slider: make it focusable on TV, handle DPAD_LEFT/RIGHT as accumulating seek (accelerate on hold: 10s -> 30s -> 60s) with an on-screen seek target preview, commit on release. Replace delay(100)+requestFocus with Modifier.focusRestorer()/FocusRequester tied to AnimatedVisibility's LaunchedEffect post-layout. Remove the dead volume state or wire a vertical-drag volume gesture (phone) / no-op (TV, where hardware volume rules).
- Risks: TV app feels broken; core leanback expectation (scrubbing) missing.
- Difficulty: Medium-Hard
- Expected improvement: TV player reaches baseline leanback quality.

H-15. Landscape lock + configChanges swallow-all prevent proper responsive behavior; zero rememberSaveable in the app

- Severity: High
 - Why: AndroidManifest.xml:30 declares configChanges="orientation|screenSize|smallestScreenSize|screenLayout|keyboard|keyboardHidden|navigation" -- the activity handles all config changes itself, which with Compose is usually fine, but combined with (a) MainActivity forcing SCREEN_ORIENTATION_LANDSCAPE for TV and (b) PlayerScreen forcing SCREEN_ORIENTATION_USER_LANDSCAPE on phones (PlayerScreen.kt:392-403), you get: no tablet layouts (device type is a binary PHONE/TV -- a 12" tablet gets stretched phone UI), no foldable posture handling, and because there's no rememberSaveable anywhere in the app (verified: zero occurrences), any config change the system does deliver (e.g., locale, uiMode) resets scroll positions, selected tab, search text -- everything. Tab state lives in the ViewModel (survives), but LazyColumn scroll state, controlsVisible, pager positions all reset.
 - Files: app/src/main/AndroidManifest.xml:30, app/src/main/java/com/example/aniflow/MainActivity.kt:30-32, app/src/main/java/com/example/aniflow/ui/player/PlayerScreen.kt:392-417, plus all screens (no rememberSaveable)
 - Root cause: Config-change handling strategy never decided; state hoisting incomplete.
 - Fix: Use rememberSaveable/rememberLazyListState() hoisted into ViewModels or saveable holders for scroll/pager state on all list screens. Add a MEDIUM/EXPANDED window-size class path (Material3 calculateWindowSizeClass) so tablets get the TV-like grid layouts instead of the phone column. Keep the player's landscape lock (it's reasonable) but also handle the case where the user backgrounds the app mid-player (orientation currently stays USER_LANDSCAPE until dispose).
 - Risks: State loss on process death/config change; poor tablet reviews.
 - Difficulty: Medium-Hard
 - Expected improvement: Survives config changes; tablet-class layout support.
-

PART 3 -- MEDIUM SEVERITY ISSUES

M-1. No Coil ImageLoader configuration -- default caches, no crossfade policy, no placeholders

AsyncImage used 32 times with no shared ImageLoader, no memory/disk cache sizing, no placeholder/error drawables (poster grids show blank boxes while loading, and broken URLs render invisible gaps). Fix: implement SingletonImageLoader.Factory on the Application class: 25% memory cache, 250 MB disk cache, crossfade(200), and add lightweight placeholder + error painters in a shared AnimePosterImage composable to replace the 32 raw call sites. Files: all screens; new AniFlowApp.kt. Difficulty: Easy. Improvement: perceived loading polish, fewer re-downloads.

M-2. Modifier.composed in hot-path glass modifiers

GlassModifiers.kt uses composed {} for glassSurface, focusGlow, darkGlassSurface -- composed is the slow path (defeats modifier skipping/reuse; officially discouraged). glassSurface is applied to nearly every card in redesign lists. Fix: convert to

plain functions taking parameters (Modifier.glassSurface(...) returning clip().background().border() directly -- none of them need composition except the animated ones; for focusGlow, migrate to Modifier.Node or accept an externally-remembered animation value). Files: ui/redesign/theme/GlassModifiers.kt:24,56,100,140. Difficulty: Medium. Improvement: measurably fewer allocations during list scroll.

M-3. filmGrainOverlay allocates a java.util.Random and a List<Offset> of up to 3000 points every frame, forever

GlassModifiers.kt:100-137: an infinite 150ms transition invalidates draw continuously; each draw allocates Random, boxes 100-3000 Offsets. Applied full-screen via AmbientBackground on every redesign screen -> permanent ~7 fps invalidation loop + GC churn even when the UI is idle, plus the three radial-gradient blob animations (AmbientBackground.kt, two more infinite transitions) redraw the whole background each frame. This is the single biggest standing battery/jank cost in the redesign flavor. Fix: pre-generate 3-4 grain textures once into ImageBitmaps and cycle them; or drop grain entirely (at 3% alpha it's nearly invisible); pause infinite animations when not visible and honor reduced-motion (ANIMATOR_DURATION_SCALE == 0). Files: GlassModifiers.kt:100-137, AmbientBackground.kt. Difficulty: Medium. Improvement: idle CPU near-zero on redesign screens; battery win.

M-4. AniListApi LRU cache accessed with synchronized from coroutines + cache key includes unhashed full query text

Works, but each key is a multi-KB GraphQL string concatenated with variables (AniListApi.kt:41), and the LinkedHashMap constructor param 100 vs removeEldestEntry > 50 mismatch shows confusion. Fix: key on (operationName, variables); guard with Mutex; single map size constant. Difficulty: Easy.

M-5. Update check downloads on every app start with cache-buster

checkUpdates appends ?t=\${currentTimeMillis} defeating HTTP caching, runs at every cold start after 3s. lastUpdateCheckTime is stored in Settings but never used to throttle. Fix: skip if now - lastUpdateCheckTime < 6h. Files: DefaultAnimeRepository.kt:436, MainScreenViewModel.kt:113-176, SettingsStore.kt. Difficulty: Easy.

M-6. AppLoader global singleton couples the ViewModel to a redesign UI component

MainScreenViewModel.kt:299 calls com.example.aniflow.ui.redesign.components.AppLoader.setLoaded(true) -- data layer -> UI-component dependency, wrong direction, and never reset to false. Fix: expose isInitialLoadComplete: StateFlow<Boolean> on the ViewModel; IntroOverlay observes it via parameter. Difficulty: Easy.

M-7. Emoji used as icons in section titles; dead accessibility on poster cards

Section headers like "(film emoji) Continue Watching", "(fire emoji) Trending Now" (RedesignPhoneHomeScreen.kt:71,113 and siblings) render inconsistently across OEM emoji fonts and are read aloud awkwardly by TalkBack. 34 images have contentDescription = null -- correct for decorative posters only if the card itself has a proper semantic label, but the poster cards' clickable has no onClickLabel/semantics with the anime title, so TalkBack reads "unlabeled, button". Fix: drop emojis for Icon(...) + text or plain text; add Modifier.semantics { contentDescription = anime.title } on each card. Files: all home screens, GlassCard.kt. Difficulty: Easy. Improvement: consistent rendering + real screen-reader support.

M-8. All text sizes in raw sp with no Material typography roles and mixed hierarchy

fontSize = 18.sp / 14.sp / 13.sp / 12.sp hardcoded per call site across screens; Type.kt defines a Typography that's barely used. Font scaling works (sp scales), but line heights are unspecified nearly everywhere -> clipped text at large font scales, and hierarchy drifts between screens. Fix: map to MaterialTheme.typography.titleMedium/bodyMedium/... and only override where genuinely needed. Difficulty: Medium (mechanical).

M-9. Touch targets below 48dp in the player

PhonePlayerControlItem (standard flavor) padding horizontal 8.dp, vertical 4.dp around 13sp text (PlayerScreen.kt:851) yields ~30x21dp targets for "-10s/+10s/Quality/Subtitles/Speed" -- well below the 48dp minimum, in the hardest-to-tap context (video playback). The < / > prev/next "icons" are ASCII text, not icons (line 666,696) -- visually off-brand and unreadable to TalkBack. Fix: Modifier.minimumInteractiveComponentSize() + real Icons.Rounded.SkipPrevious/SkipNext with contentDescriptions. Difficulty: Easy.

M-10. UpdateTakeoverScreen back-press not handled; blur scrim issues

The takeover covers the screen but doesn't intercept the system back (no BackHandler) -- back pops navigation underneath the update dialog while it stays visible over the new screen. On TV there's no dismiss affordance at all when forceUpdate=false besides "Skip" button focus. Also the blur-behind (MainScreen.kt:113-121) applies blur(20.dp) to the entire main content -- expensive on low-end devices; on API < 31 Modifier.blur silently no-ops so Android 11 users get no scrim differentiation beyond alpha. Fix: add BackHandler(enabled = updateInfo != null) { if (!forceUpdate) viewModel.dismissUpdate() }; use a dim scrim instead of blur below API 31. Files: MainScreen.kt:111-524. Difficulty: Easy.

M-11. Search UX gaps

setUpSearchDebounce requires query.length >= 2 but nothing tells the user; there's no empty-state ("No results for X") vs initial-state distinction (both are emptyList); pagination loadNextSearchPage exists but there's no genre pagination (genre select fetches one page of 15 and hasNextPage=false hardcoded, MainScreenViewModel.kt:389-411); rapid genre->query switches can interleave because onGenreSelected's launch isn't cancelled by the debounce collector's collectLatest. Fix: unified SearchUiState { Idle, Loading, Empty(query), Results(list, hasMore) }; single searchJob cancelled on each new intent; genre pagination via page plumbing. Difficulty: Medium.

M-12. Anime model reuse for AiringAnime clicks fabricates fake objects

RedesignPhoneHomeScreen.kt:95-102 (and siblings) builds Anime(id = item.mediaId, title=..., episodes = item.episode) -- where episodes is set to the airing episode number, not the total count; downstream Detail screens re-fetch by id so it's mostly harmless, but any code reading episodes pre-fetch shows wrong data. Fix: navigation only needs the id -- change onAnimeClick to (Int) -> Unit or add an onAnimeIdClick. Difficulty: Easy.

M-13. Json configured with isLenient + coerceInputValues globally

NetworkModule.kt:11-15: lenient coercion masks provider schema drift (fields silently become defaults instead of failing loudly in debug). Fine for release resilience; add a debug-only strict mode to surface drift early. Difficulty: Easy.

M-14. Ktor Android engine (HttpURLConnection-based) instead of OkHttp

ktor-client-android uses blocking HttpURLConnection under the hood; the OkHttp engine gives connection pooling, HTTP/2, and transparent gzip -- significant for the 13-request stream fan-out. redirectClient (NetworkModule.kt:28-37) is declared but appears unused (verify then delete). Fix: switch to ktor-client-okhttp; remove redirectClient if unused. Difficulty: Easy.

M-15. MainScreen composable is 869 lines mixing nav bars, 16 screen permutations, and the update dialog

Also duplicated tab lists, duplicated home-screen parameter fan-out x4 (Tv/RedesignTv/Phone/RedesignPhone all take the same 12 params). Fix: extract HomeScreenData state holder class passed as one parameter; extract AppBar/AppTvNavBar/UpdateTakeoverScreen to separate files; table-drive the tab metadata once. Difficulty: Medium. Improvement: the file becomes reviewable; adding tab #5 becomes a one-line change.

M-16. Edge-to-edge declared but insets unevenly applied

enableEdgeToEdge() is called, and redesign bottom bar uses navigationBarsPadding(), but the standard NavigationBar path and TV top padding (padding(top = 40.dp) hardcoded, MainScreen.kt:132) don't use WindowInsets -- on phones with display cutouts in landscape (player) the controls can sit under the cutout (PlayerScreen uses fixed 16-24dp paddings, no displayCutout insets, lines 602-614). Fix: Modifier.windowInsetsPadding(WindowInsets.displayCutout) on player top/bottom bars; statusBarPadding() instead of magic 40dp. Difficulty: Easy.

PART 4 -- LOW SEVERITY ISSUES (condensed)

- L-1. Icons.Default. vs Icons.Rounded. mixed in the same nav bars (MainScreen.kt:15-21,307-311) -- pick one family. Easy.
- L-2. formatTime uses String.format with default locale suppressed via @SuppressLint -- use Locale.US explicitly (PlayerScreen.kt:941-952). Easy.
- L-3. DeviceDetector caches result forever in a @Volatile var (DeviceType.kt:14-22) -- fine, but signal 3 (no touchscreen) misclassifies some Chromebooks/car headunits as TV. Consider uiMode-only + leanback. Easy.
- L-4. Episode.description/thumbnail fetched but never rendered anywhere -- either surface (nice episode list upgrade) or stop

mapping. Easy.

- L-5. SpeedSelector sets speed on the player and VM (PlayerScreen.kt:820-824) -- the VM's init also loads a default speed that races with the settings value if playback starts before DataStore returns. Consolidate: VM is the single writer, screen observes. Easy.
 - L-6. applyVideoQualityOverride sets both setMinVideoSize(matchedWidth, matchedHeight) and an explicit track override (PlayerScreen.kt:1017-1024) -- redundant; the min-size constraint can black-screen if the matched track disappears on a source switch; the override alone suffices. Also the seekTo(currentPosition) flush (line 1030) causes a visible rebuffer -- track overrides apply without seek. Medium.
 - L-7. publish_update.py embeds the release workflow with no checksum generation -- extend it alongside C-3. Easy.
 - L-8. androidx.compose.foundation:foundation added without version (BOM covers it -- OK) but inconsistent style vs catalog; move to catalog. Trivial.
 - L-9. WatchlistStore.isBookmarked reads the whole list per check while isBookmarkedFlow exists; both decode the entire JSON blob per op -- acceptable at <= few hundred items, note for Room migration. Low.
 - L-10. strings.xml contains almost nothing -- nearly every user-facing string is hardcoded in Kotlin ("Update Available", "Retry", "Subtitles", ...). Localization impossible without a sweep. Medium effort, Low urgency.
 - L-11. Kotlin 2.3.20 + AGP 9.0.1 + compileSdk 36 + Compose BOM 2026.03.01 -- bleeding-edge versions; pin known-good combos and update deliberately. Low.
 - L-12. IntroOverlay calls onFinish() during composition (IntroOverlay.kt:117-120) -- a state write in a parent during child composition; move into a LaunchedEffect(introState). Easy.
 - L-13. MainActivity intro: showIntro state is lost if the activity recreates mid-intro (no rememberSaveable) -- intro replays. Easy.
 - L-14. Manifest polish: consider android:appCategory="video"; verify leanback banner size 320x180 for ic_banner. Trivial.
-

PART 5 -- FOCUSED REVIEWS

UI/UX Review (per surface)

Home (all 4 variants): Empty-state only via row omission -- a fresh offline install shows fallback fake data (H-6). No pull-to-refresh anywhere. No skeleton/shimmer loaders -- a single centered CircularProgressIndicator gates the whole page (MainScreen.kt:153-155) even though batch-1 data could render progressively. Emoji headers (M-7). Four near-identical home implementations (~2,000 LOC total) -- extract shared row components.

Detail: Banner gradient treatment is good. Bookmark toggle exists in both variants but there's no episode-progress indication in the episode list (history data isn't passed in). Back button is custom instead of IconButton (loses ripple/48dp guarantee). "No episodes found" message missing (H-12).

Player: Strongest and weakest screen. Double-tap seek + indicators: good. Slider seek-per-tick: bad (H-3). Buffering overlay dims the whole video at 30% black -- prefer a spinner only. Controls auto-hide at 5s fixed -- restart the timer on any interaction (currently interacting with the slider doesn't reset it; the LaunchedEffect(controlsVisible, isPlaying) only re-arms on those two states). Subtitle styling not configurable (size/background) -- Media3 SubtitleView styling not touched. Error screen offers "Switch Quality" -- good touch.

Search/Browse: Debounce 300ms: good. No keyboard "search" IME action handling visible; genre chips + query mutually exclusive is sensible but not communicated. Empty vs no-results states indistinguishable (M-11).

Library: Watchlist-only; history lives on Home. No sort/filter. Removal only by toggling in Detail.

Settings: Duplicated x3 (Phone/Tv/Settings screens ~975 LOC combined). Ensure confirmation dialogs on clear-history/watchlist. Theme switch applies via global-var mutation (C-6) -- visible mid-frame color tearing risk.

Motion: Bottom-bar liquid capsule (MainScreen 247-359) is genuinely nice; spring params reasonable. But lastTab update via delay(280) is a race against the spring -- derive direction from target vs current animated value instead. Intro overlay total cold-start cost: video decode + full app blocked behind AppLoader -- consider skipping intro on subsequent launches (it currently plays every launch).

Dark/Light: Light mode is half-implemented (C-6): darkColorScheme with light values, glass tokens tuned only for dark, player hardcodes Color.Black background (correct) but selectors/dialogs use dark glass over light theme. Either finish light mode or

remove the option.

Edge-to-edge / cutouts: See M-16. Gesture nav: Player hides system bars with swipe-transient behavior: correct. Bottom-bar 16dp above nav bar: fine.

Performance Review (summary)

Recomposition: film grain + ambient blobs = permanent invalidation (M-3, biggest). composed modifiers (M-2). MainScreen collects 20 StateFlows at top level -- every tick of any one recomposes the switch host; passing 12 lists down as parameters to each home screen means one list update recomposes the whole home (mitigate via state-holder + @Immutable wrappers; mark models @Immutable to help skipping). Startup: intro videos decode at launch; update check at +3s; batched loads reasonable. Memory: 50-entry JSON history rewritten wholesale each 15s heartbeat (acceptable churn but see C-4). Network: 13-way stream fan-out (H-8), no OkHttp pooling (M-14), no image loader tuning (M-1). APK: 51 MB videos (H-10), unshrunk Ktor (C-8), unused deps (H-9).

Video Player Review (delta not already covered)

- Buffering config (PlayerScreen.kt:146-155): 50s min / 120s max buffer with setPrioritizeTimeOverSizeThresholds(true) -- very aggressive for mobile data (can prefetch ~120s of 1080p ~ hundreds of MB per session). Recommend 15s/50s defaults, or a "Data saver" setting.
- Initial bitrate estimate 15 Mbps (line 123) forces top-rung HLS start on slow networks -> long initial stall. Use the default or per-network estimates.
- setAllowCrossProtocolRedirects(true) (line 130): allows HTTPS->HTTP redirect -- combined with cleartext permitted this downgrades stream privacy. Prefer disallowing and fixing sources.
- Subtitle mime detection by .ass substring (line 288): .srt files get labeled VTT and fail to parse; add MimeTypes.APPLICATION_SUBRIP for .srt.
- Audio: handleAudioFocus=true handles ducking -- OK. Becoming-noisy (headphones unplug) not handled: add setHandleAudioBecomingNoisy(true) on the builder.
- Emulator decoder reordering (lines 157-176) -- clever, harmless in prod, keep.

Security Review (delta)

- No secrets/API keys found in source (AniList is keyless; provider is public) -- good.
- The "raye" XOR "decryption" key aproxy2026 (AniLightProvider.kt:403) is not a secret and can't be -- fine, but don't rely on it for anything security-relevant.
- AdBlocker.filterHeaders is defined but never called (grep confirms only shouldBlock used) -- dead code.
- DataStore contents (watch history) are plaintext -- acceptable for this data class; no auth/tokens exist in the app.
- allowBackup="true" + default rules: watch history/settings restore to new devices -- acceptable; document intent in backup_rules.xml (currently default template).
- Legal note: the app streams third-party-scraped content; that carries takedown and Play-distribution risk that no code change fixes -- flagged for awareness, not a code issue.

Dependency Review

Remove (unused, verified): tv-foundation, tv-material, lottie-compose, orbital, konfetti-compose, haze, haze-materials, androidx-palette, media3-session (unless doing H-2's MediaSession), ktor-client-logging. Replace: ktor-client-android -> ktor-client-okhttp. Keep/verify versions: media3 1.5.1 is fine; Coil 3.0.4 -> 3.1+ has fixes; serialization 1.7.3 OK with Kotlin 2.3.x (verify matrix).

Build Review

Single module is appropriate at this size. Add: buildConfig = true (needed for H-7), ABI splits or App Bundle if Play distribution ever happens, nonTransitiveRClass=true (check gradle.properties), and a CI workflow (assembleRelease + lint + testDebugUnitTest) -- currently nothing validates a commit compiles.

PART 6 -- BUG HUNT (crash/race candidates ranked)

1. Race: concurrent WatchHistoryStore.saveProgress read-modify-write (C-4) -- data loss, reproducible by pausing right at the

15s heartbeat.

2. Stale closure: player listener currentEp (H-1) -- wrong-episode progress writes.
 3. Lifecycle: background playback with no session (H-2); orientation stays landscape if process killed in player.
 4. Rapid clicking: tapping an episode twice quickly pushes two Player entries onto the Nav3 backstack (Navigation.kt backStack.add with no debounce/distinct guard) -- two ExoPlayers created, first leaks audio until dispose. Add a backStack.lastOrNull() != key guard.
 5. State restoration: zero rememberSaveable (H-15) -- process death in Detail: Nav3 backstack keys are @Serializable -- verify rememberNavBackStack restoration actually restores Detail args; Main tab resets to 0.
 6. updateInfo!! double-bang (MainScreen.kt:518-522): if dismissUpdate() lands between the null-check recomposition and the click lambda executing, updateInfo!! throws NPE. Capture val info = updateInfo ?: return locally.
 7. streamingSources!! same pattern in PlayerScreen.kt:798-816.
 8. Slow internet: 15 Mbps initial estimate + 50s min buffer = multi-minute first-frame stalls on 3G (perceived hang; users kill app).
 9. Offline cold start: fake fallback catalog rendered as real (H-6); tapping fallback items -> Detail refetch fails -> infinite spinner (H-12 combination).
 10. IME race: search field Enter handling not visible in browse screens -- verify CJK composition guard if Enter-to-search is added.
 11. onKeyEvent on Box with focusable() intercepts BACK when controls visible on phones too (PlayerScreen.kt:447-450) -- physical-back on phone hides controls instead of exiting; with gesture nav the predictive-back animation will fight this; no BackHandler coordination.
 12. IntroOverlay on TV: pointerInput touch-consumption does nothing for D-pad -- key events during intro fall through to the app underneath (composed simultaneously at alpha 0 and focusable -- focusProperties{canFocus=false} only applied for the update dialog, not intro). D-pad presses during intro can trigger invisible UI.
-

PART 7 -- TOP IMPROVEMENTS (ranked, grouped)

Critical (do first): C-1 delete/replace broken tests | C-2 release signing | C-3 update-channel verification (sha256 + cert check + kill cleartext) | C-4 atomic history writes | C-5 remove Big Buck Bunny fallback | C-9 purge APKs/logcats from git | H-1 stale currentEp fix | Bug-4 backstack double-push guard | Bug-6/7 remove !! on updateInfo/streamingSources.

High: C-6 theme migration | C-7 DI/composition root | C-8 proguard cleanup | H-2 lifecycle pause + PiP or MediaSession | H-3 slider onValueChangeFinished | H-4 failover position retention + mid-play failover | H-5 repository cache unification | H-6 SectionState + kill fake fallback data | H-8 stream fan-out restructure (async/awaitAll, semaphore, watch-doc cache, deterministic ranking) | H-9 remove 10 unused deps | H-10 move/recompress intro videos | H-11 flavor source-set isolation | H-12 DetailViewModel | H-13 match-confidence + manual mapping UI | H-14 TV D-pad scrubbing | H-15 rememberSaveable + window size classes.

Medium: M-1 Coil ImageLoader + placeholders | M-2 de-composed modifiers | M-3 film-grain/ambient rewrite (pause-when-idle, pre-baked grain) | M-4 API cache hygiene | M-5 update-check throttle | M-6 AppLoader inversion | M-7 emoji->icons + card semantics | M-8 typography roles | M-9 48dp targets + real skip icons | M-10 BackHandler on update takeover | M-11 SearchUiState machine + genre paging | M-12 id-based navigation callbacks | M-14 OkHttp engine | M-15 MainScreen decomposition | M-16 insets/cutouts | buffering config sanity | .srt mime support | setHandleAudioBecomingNoisy(true) | progress indication in Detail episode list | pull-to-refresh on home | skeleton loaders.

Low: L-1...L-14 as listed | subtitle appearance settings | data-saver toggle | dead redirectClient/volume-state/filterHeaders removal | localize strings | CI workflow | @Immutable model annotations.

PART 8 -- IMPLEMENTATION ROADMAP

Phase 1 -- Critical correctness & security

1. C-9 (repo purge) -> 2. C-2 (signing) -> 3. C-3 (verified updates, cleartext off) -> 4. C-1 (tests deleted, 2 real VM tests added) -> 5. C-4 (atomic history) -> 6. H-1 (stale closure) -> 7. C-5 (fallback removal + real error UI) -> 8. Bug-4/6/7 (backstack guard, !! removal).

Exit criterion

Phase 2 -- Performance & player

9. H-3 (slider) -> 10. H-2 (lifecycle pause + PiP) -> 11. H-4 (failover) -> 12. H-8 (stream resolution rewrite) -> 13. M-3 (ambient/grain) -> 14. M-1 (ImageLoader) -> 15. buffering/bitrate config -> 16. H-9 + H-10 (deps + videos) -> 17. M-14 (OkHttp).

Exit criteri

Phase 3 -- Architecture

18. C-7 (DI root) -> 19. H-5 (repo caching) -> 20. H-6 (SectionState) -> 21. H-12 (DetailViewModel) -> 22. H-11 (flavor source sets) -> 23. M-15 (MainScreen split) -> 24. M-6, M-11 -> 25. H-13 (matching).

Exit criteri

Phase 4 -- UI polish & accessibility

26. C-6 (theme system) -> 27. M-7/M-8/M-9 (semantics, typography, touch targets) -> 28. M-10, M-16 (back handling, insets) -> 29. H-14 (TV scrubbing) -> 30. skeletons, pull-to-refresh, episode progress in Detail.

Phase 5 -- Advanced

31. H-15 (window size classes, tablet layouts, rememberSaveable) -> 32. MediaSession + background audio decision -> 33. offline home cache (Room) -> 34. subtitle styling settings -> 35. localization sweep -> 36. CI + release automation with checksums in publish_update.py.

Explicit uncertainties (stated, not invented): A build was not run in the audit environment, so version-matrix claims (Kotlin 2.3.20 ? serialization 1.7.3, AGP 9.0.1 behavior) and the exact R8 output-size impact of the keep rules should be verified with an actual assembleRelease. Not every line of the 8 home/browse screen variants (~2,500 LOC of largely parallel UI) was inspected -- issues found in the sampled variants (RedesignPhoneHomeScreen, MainScreen, DetailScreen) recur structurally in siblings, but line numbers for the siblings should be located per-file when fixing. rememberNavBackStack restoration behavior across process death (Bug-5) needs an on-device test to confirm.

This report is written to be executable issue-by-issue: each item names the exact file/lines, the mechanism of failure, and the target implementation. Recommended fixing order is the Phase list above -- Phase 1 items are independent of each other and safe to apply in parallel.